PRODUCT REQUIREMENTS DOCUMENT

Meeting Intelligence Hub

AI-Powered Transcript Analysis & Decision Tracking System

AI Sprint 2026 | Version 1.0 | April 2026

Project	Version	Sprint	Deadline
Meeting Intelligence Hub	1.0	AI Sprint 2026	April 10, 2026

1. Executive Summary

The Meeting Intelligence Hub is an AI-powered web application designed to eliminate the "Double Work" problem that plagues modern teams. Every week, organizations hold dozens of meetings that generate hours of spoken content — yet the critical decisions, action items, and reasoning buried inside those transcripts are almost never systematically captured or made searchable.

This product ingests raw meeting transcripts, automatically extracts structured decisions and action items, and provides a natural-language chatbot interface that lets any team member instantly query what was discussed, decided, or assigned — across any number of past meetings.

Built for the AI Sprint 2026 internship evaluation, this solution demonstrates practical, real-world AI utility using a RAG (Retrieval-Augmented Generation) architecture backed by a FastAPI Python backend and a clean, responsive HTML/CSS/JS frontend.

2. Problem Statement

2.1 The Core Pain

Organizations generate enormous volumes of meeting content daily. The challenge is not a lack of transcription tools — it is the inability to efficiently extract meaning from what was said. A single one-hour meeting can produce 20+ pages of raw transcript text that no stakeholder has time to read in full.

2.2 The "Double Work" Cycle

- Team members spend significant post-meeting time asking colleagues: "What did we decide about X?"
- Action items assigned verbally are forgotten or misattributed, causing task slippage

- Strategic decisions and their rationale are lost within days of the meeting
- No searchable institutional memory exists across meetings

2.3 Business Impact

Time Lost	Estimated 30–40% of follow-up work is spent re-clarifying meeting outcomes
Task Slippage	Verbally assigned action items have a significantly lower completion rate than written ones
Knowledge Loss	Key strategic reasoning is permanently lost when not captured at the point of decision

3. Goals & Success Metrics

3.1 Primary Goals

- Build a fully functional end-to-end application that ingests transcripts and surfaces intelligence
- Demonstrate practical AI capability through RAG-powered Q&A over meeting content
- Deliver a clean, evaluator-friendly demo that showcases the core user journey in under 3 minutes

3.2 Success Metrics

Metric	Target	Measurement
Transcript Upload & Parse	< 5 seconds per file	Manual timing
Action Item Extraction Accuracy	> 85% relevant items captured	Manual review of 3 test transcripts
Chatbot Response Quality	Answers grounded in transcript content	Citation verification
End-to-End Demo Flow	Completes in < 3 minutes	Screen recording review
API Uptime During Demo	100% — no crashes	Live demo observation

4. Scope

4.1 In Scope (Must Build)

- **Transcript upload portal supporting .txt and .vtt file formats**
  - Drag-and-drop or file browser interface
  - File type validation with error messages

  - Post-upload summary: filename, word count, speaker count

- **Decision & Action Item Extractor**
  - AI-powered parsing to identify Decisions vs. Action Items
  - For each action item: Who, What, By When
  - Structured table display with CSV export option

- **RAG Chatbot Interface**
  - Natural language Q&A over uploaded transcripts
  - Cross-meeting context awareness
  - Source citation: which meeting + which section answered the query

- **Dashboard Home Page**
  - List of uploaded meetings with quick stats
  - Total action items, meeting count, overall status

4.2 Out of Scope (Will Not Build)

- Sentiment analysis and tone visualization (Feature 4 from original spec)
- User authentication / login system
- Real-time audio transcription — text transcripts only
- Email notification system
- Mobile native app

Note: Sentiment analysis is deliberately excluded to protect delivery quality within the 2-day sprint window. The three core features (upload, extraction, chatbot) will be polished and fully functional rather than five features half-built.

5. Feature Requirements

5.1 Feature Priority Matrix

Feature	Description	Priority
Transcript Upload Portal	Drag-and-drop upload for .txt/.vtt files with validation and post-upload summary	**Must Have**
Decision Extractor	AI extraction of decisions made in the meeting, displayed in a structured table	**Must Have**
Action Item Extractor	Extract Who / What / By When for every task assigned, with CSV export	**Must Have**
RAG Chatbot	Natural language Q&A across all uploaded transcripts with source citations	**Must Have**

Feature	Description	Priority
Dashboard Home	Landing page with meeting list, stats, and navigation to detail views	**Must Have**
Meeting Detail View	Per-meeting page showing extracted data + chatbot panel	**Must Have**
CSV Export	Download decisions and action items as a CSV file	**Should Have**
Multi-file Upload	Upload multiple transcripts in one session	**Should Have**
Sentiment Analysis	Per-speaker tone analysis and visual timeline	**Won't Have**
Email Notifications	Alert employees when action items are assigned	**Won't Have**

5.2 Feature 1 — Transcript Upload Portal

User Story

As a team member, I want to upload my meeting transcript so that the system can analyse it and surface the key information I need.

Acceptance Criteria

- User can drag-and-drop or browse to select .txt or .vtt files
- System rejects unsupported formats with a clear error message
- After upload, system displays: filename, detected word count, estimated speaker count
- Multiple files can be uploaded in one session and grouped under a project name
- Upload completes and feedback is shown within 5 seconds

5.3 Feature 2 — Decision & Action Item Extractor

User Story

As a project manager, I want the system to automatically identify every decision and action item from the transcript so I don't have to read 20 pages of dialogue.

Acceptance Criteria

- System differentiates between Decisions (agreed outcomes) and Action Items (assigned tasks)
- Each Action Item includes: Assignee name, Task description, Deadline (or 'Not specified' if absent)
- Results displayed in a clean, readable table on the Meeting Detail View
- User can download results as a CSV file
- Extraction completes within 15 seconds for a standard 1-hour transcript

5.4 Feature 3 — RAG Chatbot Interface

User Story

As a stakeholder, I want to ask questions in plain English about what happened in our meetings and get accurate, cited answers — without reading the transcripts myself.

Acceptance Criteria

- Chatbot accepts natural language questions in a messaging-style interface

- Answers are grounded in actual transcript content (no hallucination of facts not present)
- Every answer includes a citation: which meeting file and approximate section it came from
- System can answer cross-meeting questions (e.g., 'What did Finance say across all three meetings?')
- Response time under 10 seconds for typical queries

6. Technical Architecture & Stack

6.1 Tech Stack

Layer	Technology	Rationale
Backend API	FastAPI (Python)	Async-friendly, auto-generates OpenAPI docs, fast to scaffold
AI / LLM	Gemini 3 Pro via Google Antigravity	Built-in, free, capable of long-context reasoning
Embeddings & RAG	sentence-transformers + FAISS	Lightweight, runs locally, no external vector DB cost
File Parsing	Python stdlib + vtt-parser	Zero overhead for plain text; VTT is standard subtitle format
Frontend	HTML5 / CSS3 / Vanilla JS	No build tools, instant setup, works in any browser for demo
Data Storage	JSON flat files (local)	Sufficient for sprint scope; no DB setup overhead
Export	Python csv module	Built-in, no dependencies

6.2 System Architecture

The application follows a clean client-server architecture:

```
[ Browser — HTML/JS Frontend ]
          |  HTTP REST calls  |
[ FastAPI Backend — Python ]
    |               |               |
[ Parser ]  [ LLM Engine ]  [ FAISS Index ]
               |
       [ Gemini 3 Pro API ]
```

6.3 API Endpoints

Method	Endpoint	Purpose	Response
POST	/api/upload	Upload and parse a transcript file	{ meeting_id, summary }

Method	Endpoint	Purpose	Response
GET	/api/meetings	List all uploaded meetings	[{ id, name, date, stats }]
GET	/api/meetings/{id}	Get full meeting detail	{ decisions, action_items }
POST	/api/chat	Send a query to the RAG chatbot	{ answer, citations }
GET	/api/meetings/{id}/export	Download action items as CSV	CSV file download

7. User Flows

7.1 Primary Flow — Upload to Insights

Step	User Action	System Response
1	Opens app — lands on Dashboard	Shows list of past meetings (or empty state for first use)
2	Clicks 'Upload New Meeting'	Opens upload modal with drag-and-drop zone
3	Drops a .txt or .vtt file	Validates format, shows filename and file size
4	Enters meeting name and clicks 'Process'	Shows progress indicator while parsing
5	Upload completes	Displays summary card: word count, speaker count detected
6	Clicks on meeting card	Opens Meeting Detail View
7	Views Decisions & Action Items table	Structured table with all extracted items
8	Types a question in the chatbot	AI returns answer with citation to source meeting
9	Clicks 'Export CSV'	Downloads action_items.csv to local machine

8. Delivery Milestones

Phase	Deliverable	Timeline	Status
Phase 1	Project scaffold — FastAPI backend, folder structure, README skeleton	Day 1 — Morning	In Progress
Phase 2	Transcript upload endpoint + file parser (.txt and .vtt)	Day 1 — Afternoon	Pending
Phase 3	LLM integration — decision and action item extraction endpoint	Day 1 — Evening	Pending
Phase 4	RAG pipeline — embeddings, FAISS index, chatbot endpoint	Day 2 — Morning	Pending
Phase 5	Frontend — Dashboard, Upload UI, Meeting Detail, Chatbot panel	Day 2 — Afternoon	Pending

Phase	Deliverable	Timeline	Status
Phase 6	Integration testing, README completion, approach document	Day 2 — Evening	**Pending**
Phase 7	Demo video recording and final GitHub submission	April 10 — Morning	**Pending**

9. Risks & Mitigations

Risk	Severity	Mitigation
LLM returns inconsistent JSON for extraction	**High**	Use strict prompt engineering with output schema; add fallback regex parser
RAG chatbot hallucinates facts not in transcript	**High**	Ground prompts explicitly in retrieved chunks; add 'not found in transcripts' fallback
FAISS index too slow for large transcript sets	**Medium**	Limit sprint demo to 5 transcripts max; note scalability improvements in approach doc
Gemini API rate limits hit during demo	**Medium**	Cache last 10 responses; pre-run demo queries before recording
VTT parsing edge cases break upload	**Low**	Test with 3 real VTT files; fallback to plain text extraction if VTT parse fails

10. Future Improvements

The following enhancements would be prioritised with additional development time:

- Sentiment & Tone Analysis — per-speaker emotional timeline visualised as colour-coded charts
- Speaker Diarisation — automatic identification of speaker names from audio metadata or context
- Real-time Transcription — live meeting transcription via browser microphone with WebSockets
- Authentication & Multi-tenancy — user accounts, team workspaces, and role-based access control
- Vector Database — migrate from FAISS to Pinecone or Weaviate for production-scale search
- Smart Reminders — automated follow-up nudges for action items approaching their deadline
- Slack / Teams Integration — push extracted action items directly into project management tools

Document Sign-off

Role	Name	Date
Project Owner	_____	April 2026

Technical Lead	_____	April 2026
Sprint Evaluator	_____	April 2026

This document is the source of truth for the Meeting Intelligence Hub sprint submission. All implementation decisions should be validated against the requirements and acceptance criteria defined herein.